Design Document

EcoVerse
July 2025



POLITECNICO
MILANO 1863

Ali Moeinkhah	10903579
Rohollan Naeijian	
Yevgenii Myasnikov	
Zhanat Argimbayeva	11059833

Professor: Giovanni Quattrocchi

1. Introduction **3**
1.1 Context and Motivations 3
1.2 Definitions, Acronyms, and Abbreviations 3
1.3 Purpose 4
1.4 Scope and Limitations 4
2. Architectural Design **5**
2.1 Overview 5
2.2 Component Diagram 5
3. Software Design **6**
3.1 Database Design 6
3.1.1 Tables 6
3.1.2 External Data 6
3.1.3 Integration 7
3.2 REST APIs 7
3.3 Dashboard Design 7
3.3.1 For Unregistered Users 8
Figure 3.1 – Login 8
Figure 3.2 – Station Preview 8
Figure 3.3 – Random Station Location 9
Figure 3.4 – Station Data Sample 9
3.3.2 For Registered Users 9
Figure 3.5 – Random Station Location 10
Figure 3.6 – Station Data Availability 10
Figure 3.7 –Sensor Data for Selected Station 11
Figure 3.8 – Combined Data Table 11
Figure 3.9 – Average Values 12
4. System Architecture **12**
4.1 High-Level Architecture Overview 12
4.2 Data Flow Description 13
4.3 Architecture Diagram Components 13
4.4 Modularity and Deployment 13
Figure 4.1 – System Architecture Diagram 14
5. Constraints **14**
5.1 Technical Constraints 14
5.2 Data-Related Constraints 15
5.3 Legal and Organizational Constraints 15
5.4 External Tool Dependencies 15
6. Test Plans **15**
7. Bibliography **16**
7.1 Open Data Sources 16
7.2 Key Tools, Libraries, and Frameworks 16
7.3 System Diagrams and Internal Deliverables 17

7.4 Institutional and Academic References.....17
7.5 Acronyms and Abbreviations.....17

1. Introduction

1.1 Context and Motivations

Air pollution is a major environmental and health issue, particularly in potentially industrialized and populated areas, like Lombardy, Italy. The growing demand for transparent air quality data, real-time air quality information, and spatially resolved air quality data has led to the realization of this project: "**Air Quality Analysis using Dati Lombardia".**

This project focuses on developing a client-server web application that provides environmental sensor data in an intuitive way. By working with open data from the Dati Lombardia portal with online mapping and visual geospatial tools, the project aims to develop a program that allows users to analyze air quality over time, provide filters for pollutants or geography, and help users generate relevant meaning from their data using visual dashboards.

As with all projects, there are motivations for developing this system, and they include, but are not limited to:

1. To provide data to promote environmental awareness.
2. To provide tools for pollution analysis to help with public health or urban planning.
3. To provide opportunities for researchers, educators, and the citizenry to analyze and visualize trends.
4. To develop an extensible and modular platform to host other environmental datasets in the future

1.2 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
RASD	Requirements Analysis and Specification Document
API	Application Programming Interface (for data access between backend and client)
AQI	Air Quality Index
PM10 / PM2.5	Particulate Matter with diameter < 10 or 2.5 microns
NO_2	Nitrogen Dioxide
O_3	Ozone
Flask	Python-based micro web framework
PostgreSQL/PostGIS	Relational database with spatial capabilities
JSON	Lightweight data-interchange format used in API responses
UI / UX	User Interface / User Experience
Jupyter Notebook	Web-based platform for building the frontend dashboard

Municipality / Province	Administrative divisions used for spatial filtering
Sensor	Device measuring pollutant concentrations in the environment

1.3 Purpose

This project is designed to provide a modular, open-source tool to query, analyze and visualize air quality data from the Dati Lombardia sensor network.

Intended users are:

- Citizens, to examine pollution levels in their region.
- Researchers investigate long-term environmental trends.
- Policy makers, to facilitate data-driven choices for public health.
- Educators, to deliver real-time data to students.

The system can provide:

- Filtering by pollutant levels (e.g., PM10, NO_2)
- Temporal and spatial queries
- Visual displays (maps, tables, graphs) by user customization
- Exporting capabilities (CSV, PNG)
- Optional personalization and user roles for the system (e.g., Admin, Analyst)

1.4 Scope and Limitations

Scope:

- The scope of this system involves the visualization of air quality (PM10, NO2, O3, etc.) at the municipality/province level.
- An interactive web dashboard has been developed using Jupyter Notebook (front-end), Flask (back-end), and PostgreSQL/PostGIS (database).
- Querying and exporting of pollutant data by pollutant, location, and date are available.
- Saving of filters, views, and favourite sensors is also supported.

Limitations:

- Forecasting and real-time weather data are not included.
- No mobile version is optimized in this version.
- There are limitations on the number of concurrent users and local deployment, which is currently Linux-based.
- The datasets are limited to the Lombardy region only.
- The system does not utilize live data feeds; instead, it works from pre-downloaded CSV files.

2. Architectural Design

2.1 Overview

The architecture of the Air Quality Analysis System is a modular client-server architecture with a clear separation between the frontend (dashboard), backend (API services), and database (sensor data storage). This architecture provides an efficient backend design, supports flexible architectures and scaling, and improves maintainability.
The architectural design divides system functions into three distinct layers:

- Presentation Layer (Client Dashboard)—designed using Jupyter Notebook for interactivity, as well as Python libraries for visualization.
- Application Layer (Flask Backend)—exposes REST API endpoints for data querying, filtering, user actions, and personalization.
- Data Layer (PostgreSQL/PostGIS Database)—used to store the air quality measurements, sensor metadata, user options, and logs of actions.
- The layers interact through standard HTTP/JSON protocols, and the components are loosely coupled, which allows for independent deployment and testing of the layers.

This object-oriented architecture has established:

- Provable efficiency in querying a large data set on air quality.
- Visualizing geospatial pollution trends.
- Provision for advanced features such as saved filtering, favourite sensors, and admin controls.
- Provision for future development that may improve usefulness by combining other environmental data sets or services, for example, traffic and weather prediction models.

2.2 Component Diagram

The architecture consists of the following components:

Client (Frontend - Dashboard):

The client was built in Jupyter Notebook, which employed:

- Plotly for charts and time series
- Folium for interactive maps
- ipywidgets for filtering dynamically

Key Features:

- Filter by pollutant, time, and location
- Exports graphs and maps as PNG or CSV
- No need to register for basic properties

Backend (Application Server - Flask):

Provides RESTful API endpoints for querying:

- Pollutant data
- User login/registration
- Personalization (saved views, favourites)
- Admin functions (logs, user activity)

Implementing:

- Validation of input
- Processing and aggregating of data
- Authentication and roles of users

Database (PostgreSQL/PostGIS):

Stores all the structured data necessary for the system's operation, including:

- Air Quality Measures (e.g., PM10, NO_2)
- Sensor metadata (location, type)
- Administrative boundaries (municipalities, provinces)
- Profiles of users, saved filters, system logs

Capabilities:

- Geospatial queries with PostGIS

External Tools:

Dati Lombardia

- Source of Pollutant data(CSV download or API)
- For development and deployment we used **VSCode, GitHub, Docker**.

3. Software Design

3.1 Database Design

3.1.1 Tables

- **Sensors:** sensor_id, type, location, status.
- **Measurements:** id, sensor_id, timestamp, pollutant, value, status.
- **Users:** user_id, username, password_hash.
- **SavedFilters:** filter_id, user_id, filter_config.
- **Logs:** log_id, user_id, action, timestamp.

3.1.2 External Data

- Dati Lombardia CSVs for air quality and sensor metadata.
- Geoportale Lombardia for administrative boundaries.

3.1.3 Integration

- Measurements joined with sensor metadata via sensor_id.
- Spatial queries use PostGIS geometry columns.

3.2 REST APIs

Endpoint	Task
/api/login	Authenticate user
/api/query	Filter measurements by pollutant, date, location
/api/save-filter	Save user filters
/api/export	Export results CSV/PNG

3.3 Dashboard Design

The Dashboard design differs significantly for registered and unregistered users. Unregistered users are presented with a limited view of the data including login function, station data preview, random station locations and limited data sample (Fig. 3.1 - 3.4).

3.3.1 For Unregistered Users

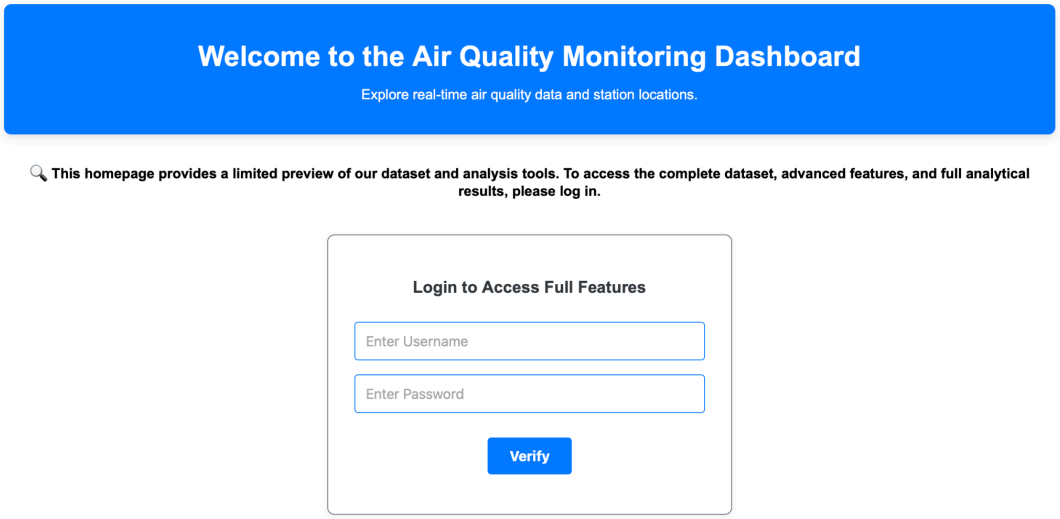


Figure 3.1 – Login

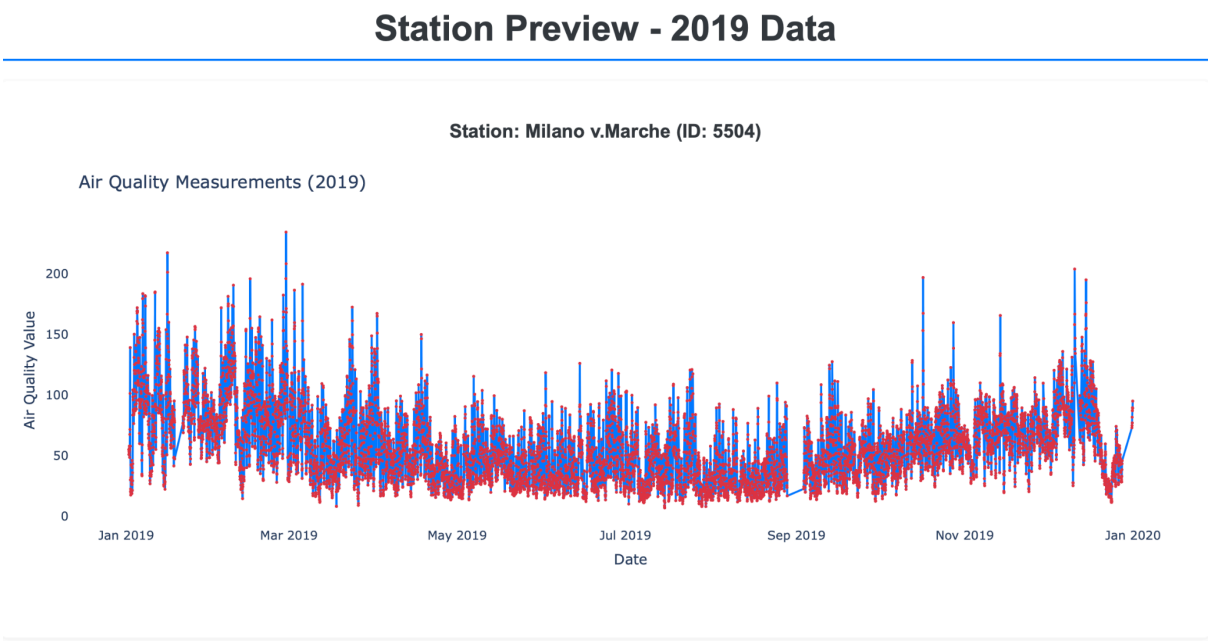


Figure 3.2 – Station Preview

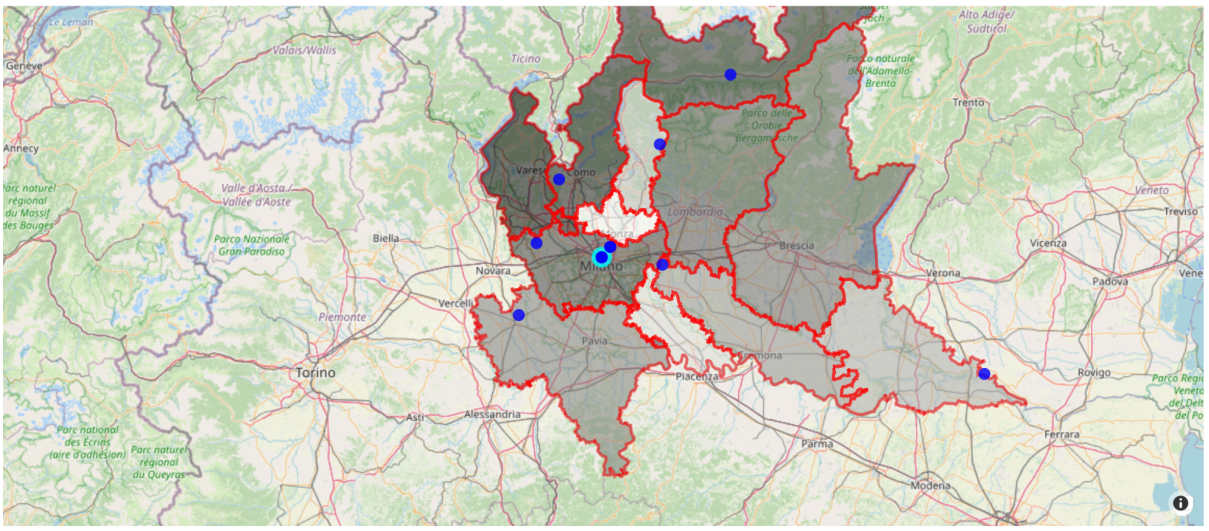


Figure 3.3 – Random Station Location

Station Data Sample (2019)

Data	Id	Idoperatore	Idsensore	Stato	Valore
filter data...					
2019-12-31 20:00:00	2796355	1	5504	VA	95
2019-12-31 20:00:00	2796355	1	5504	VA	95
2019-12-31 19:00:00	2796029	1	5504	VA	89.1
2019-12-31 19:00:00	2796029	1	5504	VA	89.1
2019-12-31 18:00:00	2795703	1	5504	VA	89.5
2019-12-31 18:00:00	2795703	1	5504	VA	89.5
2019-12-31 17:00:00	2795377	1	5504	VA	87.1
2019-12-31 17:00:00	2795377	1	5504	VA	87.1
2019-12-31 16:00:00	2795051	1	5504	VA	77
2019-12-31 16:00:00	2795051	1	5504	VA	77

« ‹ 1 / 2 › »

Figure 3.4 – Station Data Sample

3.3.2 For Registered Users

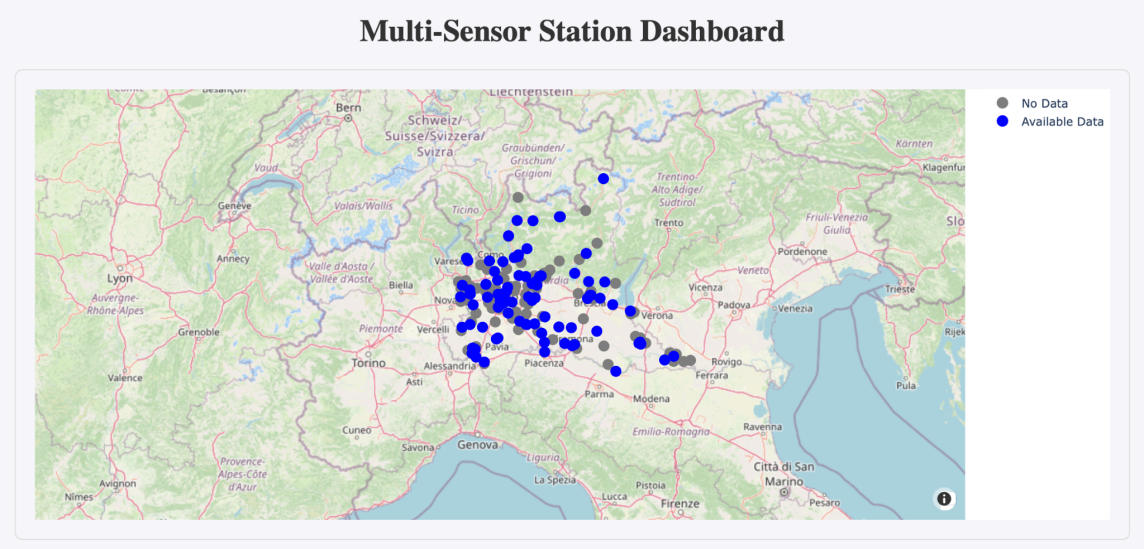


Figure 3.5 – Random Station Location

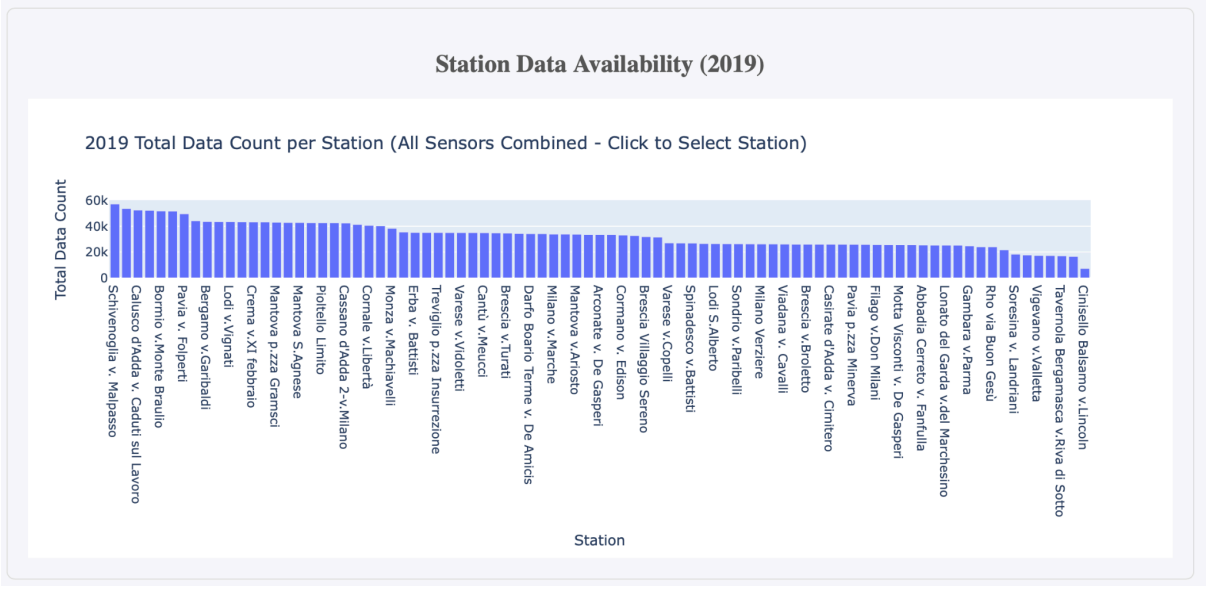


Figure 3.6 – Station Data Availability

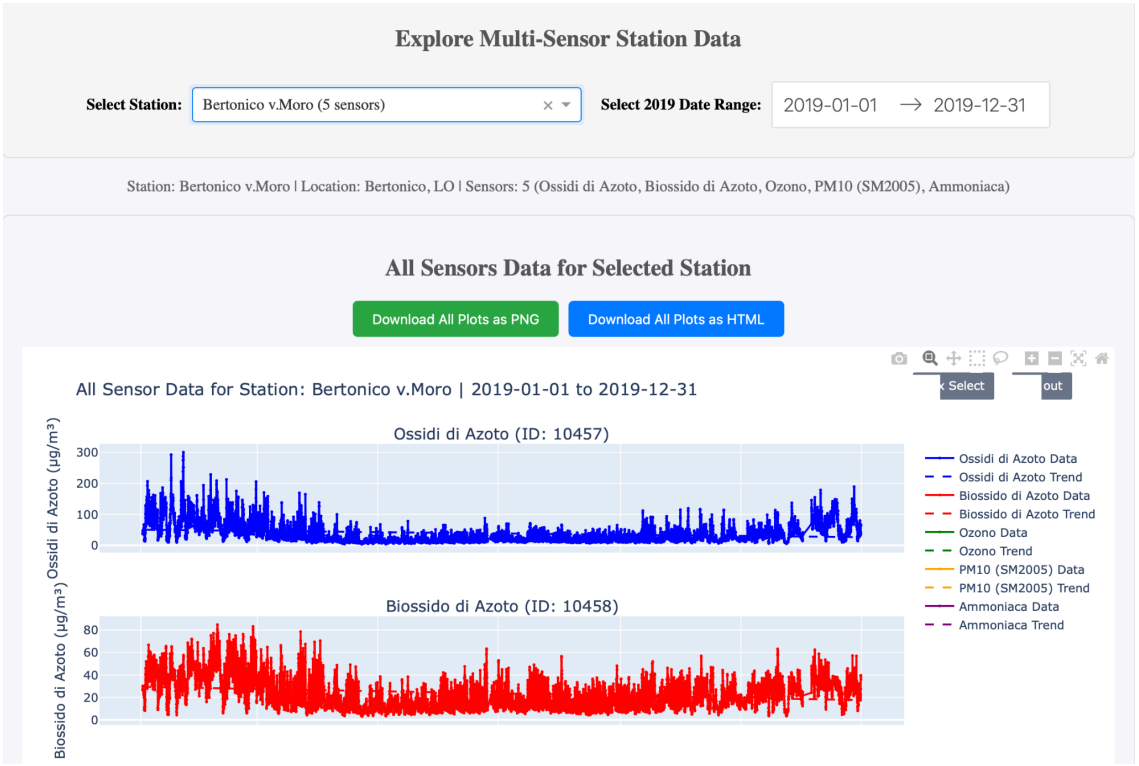
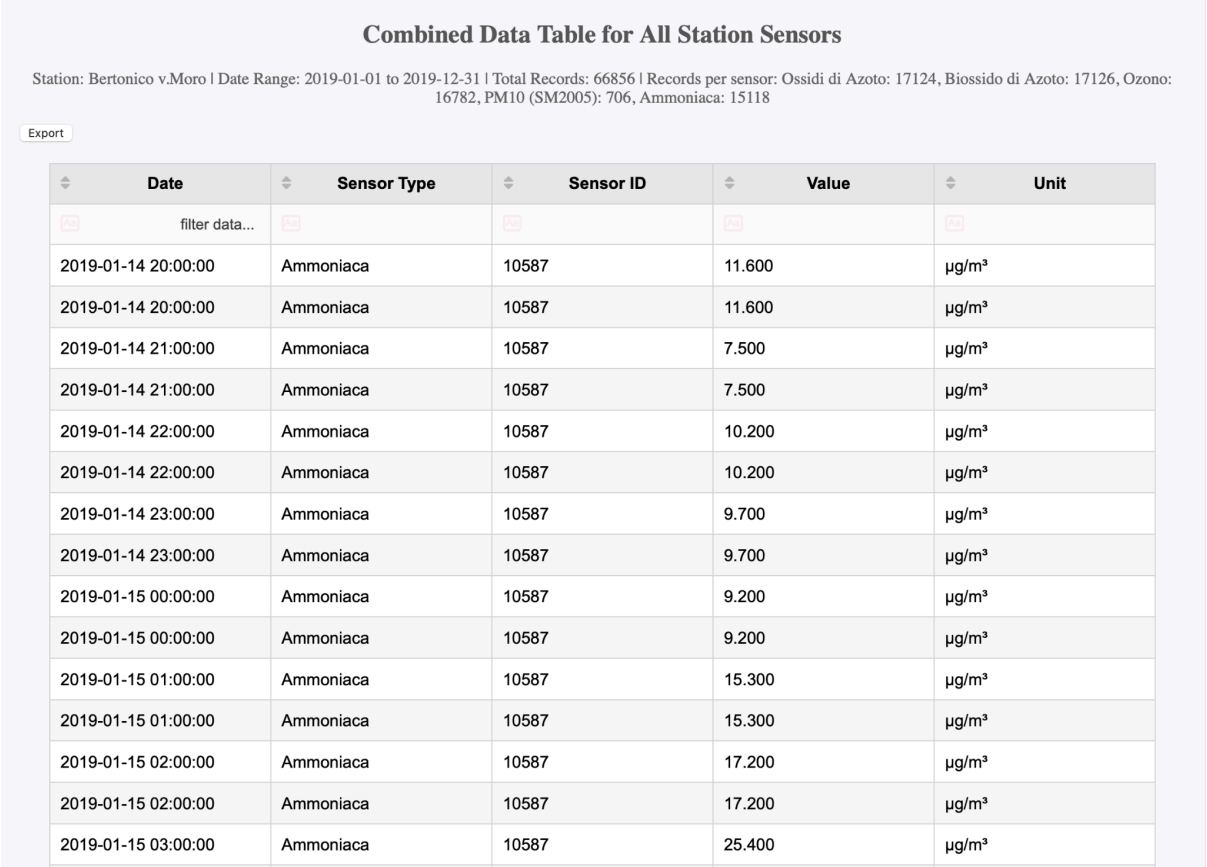


Figure 3.7 –Sensor Data for Selected Station



Date	Sensor Type	Sensor ID	Value	Unit
filter data...				
2019-01-14 20:00:00	Ammoniaca	10587	11.600	µg/m³
2019-01-14 20:00:00	Ammoniaca	10587	11.600	µg/m³
2019-01-14 21:00:00	Ammoniaca	10587	7.500	µg/m³
2019-01-14 21:00:00	Ammoniaca	10587	7.500	µg/m³
2019-01-14 22:00:00	Ammoniaca	10587	10.200	µg/m³
2019-01-14 22:00:00	Ammoniaca	10587	10.200	µg/m³
2019-01-14 23:00:00	Ammoniaca	10587	9.700	µg/m³
2019-01-14 23:00:00	Ammoniaca	10587	9.700	µg/m³
2019-01-15 00:00:00	Ammoniaca	10587	9.200	µg/m³
2019-01-15 00:00:00	Ammoniaca	10587	9.200	µg/m³
2019-01-15 01:00:00	Ammoniaca	10587	15.300	µg/m³
2019-01-15 01:00:00	Ammoniaca	10587	15.300	µg/m³
2019-01-15 02:00:00	Ammoniaca	10587	17.200	µg/m³
2019-01-15 02:00:00	Ammoniaca	10587	17.200	µg/m³
2019-01-15 03:00:00	Ammoniaca	10587	25.400	µg/m³

Figure 3.8 – Combined Data Table

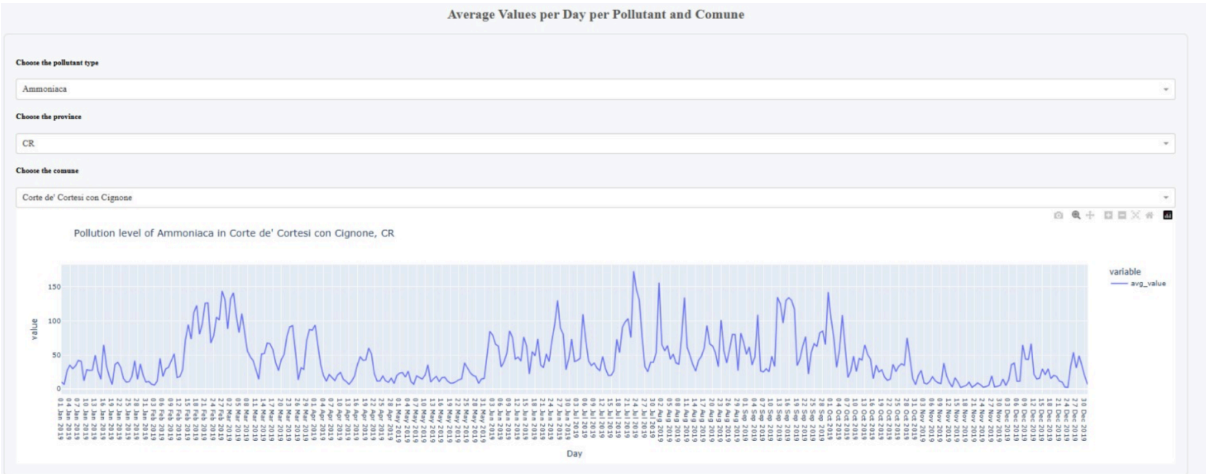


Figure 3.9 – Average Values

4. System Architecture

This section provides an overview of the technical architecture of the system. It describes the core components and the interactions between the client-side interface, backend server, and database. The architecture is designed to support scalability, modularity, and clear separation of concerns.

4.1 High-Level Architecture Overview

The system follows a client-server architecture, composed of the following three main layers:
Client Layer (Frontend – Dashboard)

- Built using Jupyter Notebooks and Python libraries such as Plotly, Folium, and ipywidgets.
- Provides an interactive interface for users to query data, apply filters, and visualize results.
- Communicates with the backend via HTTP requests (REST API).
- Includes optional login interface, map tools, personalization settings, and data export options.

Server Layer (Backend – Web API)

- Developed using the Flask web framework in Python. Exposes a set of RESTful API endpoints to handle requests from the client.
- Handles input validation, business logic, authentication, and formatting responses in JSON.
- Manages features such as login, registration, filter saving, and system logging.

Data Layer (Database – PostgreSQL/PostGIS)
Stores structured data related to:

- Sensor metadata (location, type, status)
- Air quality measurements (timestamp, pollutant, value)
- User profiles and personalization settings
- Logs and admin actions
- Uses the PostGIS extension to enable spatial queries and geographic filtering.

4.2 Data Flow Description

- The user accesses the dashboard (either anonymously or with login).
- The user submits a query (e.g., time range, pollutant, location).
- The request is sent to the Flask backend via RESTful API.
- The backend validates the input and queries the PostgreSQL/PostGIS database.
- Retrieved data is sent back to the client in JSON format.
- The dashboard displays results using charts, maps, and tables.
- If logged in, the user can save filters, export data, or customize map views.

4.3 Architecture Diagram Components

The architecture includes the following components (as shown in the architecture diagram):

- Client: Jupyter dashboard interface with widgets and map/chart tools.
- Web Server: Flask application with API routes, authentication logic, personalization handler, admin logic, and error logger.
- Database: PostgreSQL with PostGIS extension, storing sensor data, user preferences, logs, and saved filters.
- External Dataset: Dati Lombardia open data sources (CSV or API).
- Support Tools:
- VSCode: IDE used for development
- GitHub: Code repository for version control and collaboration
- Docker: Containerization of application for portability and deployment

4.4 Modularity and Deployment

- Each layer of the architecture is modular and can be maintained independently.
- The backend server and database are assumed to run in a local or hosted Linux environment with Python, PostgreSQL, and Flask stack.
- Docker may optionally be used to containerize services and simplify deployment.
- The architecture is designed to support future extensions, such as integrating additional datasets, enabling prediction models, or adding user notifications.

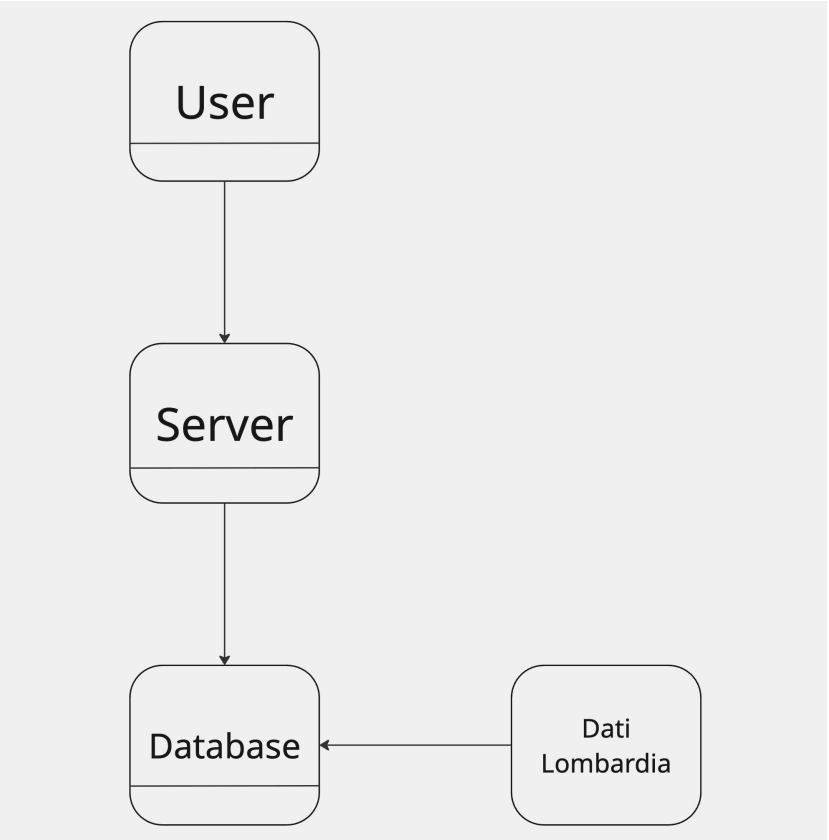


Figure 4.1 – System Architecture Diagram

5. Constraints

This section outlines the technical, organizational, and environmental constraints and dependencies that impact the design, implementation, and deployment of the system. These limitations must be acknowledged and accounted for to ensure that the project remains realistic, maintainable, and deliverable within the scope of the assignment.

5.1 Technical Constraints

TC1. Single Language and Framework Requirement
The system is developed entirely using the Python programming language. All backend functionality must be implemented using the Flask framework. This limits flexibility in choosing other languages or platforms.
TC2. Notebook-Based Dashboard
The front-end dashboard is required to be built using Jupyter Notebooks. This imposes constraints on interactivity and responsiveness compared to full web applications.
TC3. Limited Concurrency
The system is not optimized for high-traffic scenarios. Only a limited number of concurrent users are expected (e.g., academic testing), and scalability is not a primary requirement.
TC4. Local Execution Preference
Although the architecture supports deployment via Docker, the default execution environment is assumed to be a local Linux system, not a cloud-based or enterprise-grade hosting environment.

TC5. Absence of Real-Time Data Streaming
The system uses pre-downloaded CSV datasets from Dati Lombardia. It does not support real-time streaming of air quality data.

5.2 Data-Related Constraints

DC1. External Dataset Dependency
The project depends entirely on open data from the Dati Lombardia portal. If the data format changes or access becomes restricted, the system must be updated accordingly.
DC2. Incomplete or Inconsistent Data
There may be missing measurements, uncalibrated sensors, or inconsistencies in timestamps or pollutant labels. Data cleaning and validation must be done at preprocessing time.
DC3. Static Time Frame for Evaluation
For testing and demonstration purposes, a single-month subset of sensor data is used. This limits the ability to validate long-term trends or seasonal analysis.

5.3 Legal and Organizational Constraints

LC1. Open Data Licensing
All datasets used in the system must be publicly accessible and licensed for free educational use. The project may not integrate private or commercial datasets.
LC2. User Data Handling
No sensitive personal data is collected. However, basic user credentials (username, password hash) are stored. Passwords must be securely encrypted and privacy respected.
LC3. Academic Timeline Restrictions
The system must be designed, implemented, tested, and submitted according to academic deadlines. No major extensions can be made beyond the required deliverables.

5.4 External Tool Dependencies

ET1. Third-Party Libraries
The project depends on external Python libraries (e.g., Pandas, GeoPandas, Folium, Plotly). If these packages change APIs or compatibility breaks, system stability may be affected.
ET2. Docker and GitHub
The team is expected to use GitHub for version control and Docker for optional containerization. Limited experience or misconfiguration in these tools may delay integration.
ET3. System Requirements
The system assumes availability of a modern browser, internet connection, Python 3.8+, and a Linux-based execution environment for local deployment. These constraints and dependencies must be considered and documented to help guide realistic design decisions, reduce project risks, and ensure that the system can be implemented within the scope and timeframe defined by the course.

6. Test Plans

Component	Test	Tool	Expected Result
REST API Endpoints	`/api/login` returns correct status for valid/invalid credentials	Manual	Correct credentials
Data query	`/api/query` returns correct JSON for given filters	Manual	Correct pollutant data returned
Data export	CSV/PNG export generates correct files	Manual	Files contain expected data

7. Bibliography

This section includes the external sources, references, and supporting materials used in the design, development, and documentation of the system. All links, datasets, diagrams, and tools listed here were either directly integrated into the system or influenced architectural and technical decisions. The section also includes internal deliverables, acronyms, and documentation standards that support transparency and traceability.

7.1 Open Data Sources

- Dati Lombardia – Air Quality Measurements Dataset URL:
  URL:https://www.dati.lombardia.it/Ambiente/Dati-sensori-aria-dal-2018/g2hp-ar79
  Description: Hourly air quality measurements from multiple monitoring stations across the Lombardy region. Used as the core input dataset for pollutant data.
- Dati Lombardia – Sensor Metadata Dataset
  URL: https://www.dati.lombardia.it/Ambiente/Stazioni-qualit-dell-aria/ib47-atvt
  Description: Geographical and technical metadata of air quality sensors used to support spatial queries, data integration, and visualization.
- Geoportale Lombardia – Administrative Boundaries
  URL: https://www.geoportale.regione.lombardia.it/download-ricerca
  Description: GIS shapefiles of provinces, municipalities, and regional boundaries used for spatial aggregation and map layering.

7.2 Key Tools, Libraries, and Frameworks

Programming and Development

- Python – Core programming language
- VSCode – Source code editor with extensions for Python and Docker
- Git & GitHub – Version control, issue tracking, and collaborative development

Web and Backend Development

- Flask – Web framework to develop RESTful APIs in Python
- Docker – Containerization for reproducible and portable development environments

Database and Spatial Tools

- PostgreSQL– Relational database system
- PostGIS – Extension for geospatial data storage and querying

Data Analysis and Visualization

- Pandas – Dataframe manipulation and processing
- GeoPandas – Spatial joins and geo-data processing
- Plotly – Interactive graph generation (line, bar, pie charts)
- Folium – Leaflet-based map rendering in Python
- ipywidgets – For interactivity in Jupyter dashboard
- Matplotlib, Seaborn – Static charts and plots (optional)

Dashboard Interface

- Jupyter Notebook – Interactive frontend used to develop and deploy the dashboard

7.3 System Diagrams and Internal Deliverables

- System Architecture Diagram – Illustration of three-layer system: Client, Backend, Database (Section 6)
- Use Case Diagram – Actors, interactions, and authentication flows (Section 3.2)
- Deployment Diagram – Dockerized modular setup, local deployment flow (Section 6.3)
- Functional Requirements Table – Modular requirement list from F1.1 to F6.3 (Section 4)
- Use Case Descriptions – Step-by-step breakdown of user and admin interactions (Section 3.3)
- Obstacle and dependence Table – Technical and organizational limitations (Section 7)

7.4 Institutional and Academic References

- SE4GEO Project Assignment 2025 – Course brief and deliverables timeline from Politecnico di Milano
- Scrum Handbook (WeBeep) – Reference for agile software development methodology
- University Academic Calendar – Milestone tracking and project deadlines
- GitHub Project Repository – Private repository for team collaboration and source management
- Python Guidelines – Style guide for writing clean and readable Python code

7.5 Acronyms and Abbreviations

Acronym	Description
API	Application Programming Interface
CSV	Comma-Separated Values
MVP	Minimum Viable Product
REST	Representational State Transfer
UI	User Interface
DB	Database

IDE Integrated Development Environment
CRUD Create, Read, Update, Delete
GIS Geographic Information System
JWT JSON Web Token (authentication method)
WSGI Web Server Gateway Interface

This section serves as a reference index to all relevant technical tools, data sources, and documentation components referenced throughout the project and supports verification, reproducibility, and compliance with academic standards.